

主 题:	Software and Systems Modeling - Decision on Manuscript ID SOSYM-25-00005206	
发件人:	"Software and Systems Modeling" <onbehalf@manuscriptcentral.com>	2026-1-22 18:30:05
收件人:	zhentian1@buaa.edu.cn	
抄 送:	yilongyang@buaa.edu.cn, runkunzhang@buaa.edu.cn, zhentian1@buaa.edu.cn, weiru.wang@bjut.edu.cn	

22-Jan-2026

Dear Mr. Tian:

I write you in regards to manuscript # SOSYM-25-00005206 entitled "An Agent-Oriented Framework to Support Rapid Early-Stage Object-Oriented Design" which you submitted to the journal of Software and Systems Modeling.

In view of the criticisms of the reviewers found at the bottom of this letter, I regret to inform you that your manuscript cannot be accepted for publication in the journal of Software and Systems Modeling.

Thank you for considering the journal of Software and Systems Modeling for the publication of your research. I hope that the comments of the reviewers will help you to improve the research described in the manuscript.

Sincerely,
Professor Benoit Combemale
Editor in Chief, Software and Systems Modeling
benoit.combemale@sosym.org

Editor's Comments to Author:

Editor

Comments to the Author:

Thank you for your patience with the reviews. I can only apologize - one of the reviewers experienced significant personal difficulties during the review process.

As you will see, we have received two consistent reviews for your paper. Both reviews make note of the interesting topic, and the use of a combination of MDE and LLM-based generation is intriguing (though one reviewer suggests it may not be novel). However, I take note of the criticisms around how this builds on your previous work -- this really needs to be made more clear -- and the threats to validity around the metrics and experiments.

Stepping back, if you were to revise the paper, I would make three suggestions:

1) The presentation of MDE/MDA is not current, it reflects thinking from the late 2000s/early 2010s.

2) Consider comparisons using a model transformation language, not (only) OCL. While it is possible to declaratively specify mapping transformations in OCL, it is not always a sensible idea, as you have to write a lot of boilerplate "code" that a mapping language like ATL/ETL will do for you. You still have the overhead of writing rules, but transformation rules *are* the right thing to use for this process, as not all of the patterns you are encountering are purely declarative.

3) Try to support external evaluation, not just examples and metrics that you control yourselves. This will help give some arguments towards generality and repeatability.

So, overall, there is promise in the work, but in its current form, I unfortunately cannot recommend it for acceptance.

I apologize again for the delay in processing your paper, and wish you success in your research going forward.

Reviewers' Comments to Author:

Please note: Reviewers might have uploaded their review as a separate files which unfortunately are not attached to this message. To get access to these files please go to your author center in Manuscript Central and click on "Manuscripts with Decisions". Find the entry with Manuscript ID SOSYM-25-00005206 and click on the link "view decision letter" next to it. At the end of the appearing pop up window you'll find links for every attached file if exist. Please make sure that your browser does not suppress pop up windows.

Reviewer: 1

Public Comments (these will be made available to the author)

The paper proposes a method for using agents to support object oriented design. The authors propose starting with a model-driven, rules-based generation procedure to generate designs from requirements models, and subsequently applying a rules-based analysis agent aiming to reliably generate a starting point of a design based on existing requirements models. Following this, the authors propose quality attribute-focused prompts to LLM-based agents that leverage their knowledge-emergence capabilities to provide recommendations to iteratively improve upon the rules-based design. An evaluation is presented, comparing designs produced by graduate-level software engineering students with and without the use of agents to those produced by experts as a means to assess the agent's effectiveness. This is all done within the context of the anemic domain model design pattern.

Strengths:

- Topic of using agents to support software development is very relevant. The model-based portion of the approach and surrounding discussion are well explained.

- The approach addresses the relevant challenge of leveraging the advantages of LLMs while mitigating potential issues. The suggested prompts emphasize having LLMs produce recommendations and rationale that humans can evaluate and choose to apply. Results are promising, showing that integrating model-based and LLM-based methods in this way can improve quality.

- The evaluation is approached objectively, using design metrics to evaluate the quality of resulting designs over six samples.

Weaknesses:

- There are issues with formatting. Some examples:

- 2.2 is a very long paragraph, splitting up the concepts would make it easier to understand

- References in section 2.3

- In 4.3.2 the items in the list would be easier to follow as subsections or paragraphs

- Acronyms should be defined, for example some on page 5

- The originality and significance of the LLM-based portion would be clearer if related work discussed other LLM-based refactoring or assessment approaches. The paper titled "LLM-Driven Code Refactoring: Opportunities and Limitations" by Cordeiro et al would be a good starting point.

- In 5.5.2, the effectiveness of the method would be clearer if the quality of the initial generated design, quality after rules-based refactoring, and finally quality after LLM-based refactoring were documented. Comparison to LLM-only approaches is insightful but does not necessarily match RQ2, perhaps it could be an additional question or included as a separate table.

- The significance of the first and third contributions would be clearer if the evaluation considered use of LLMs in more detail and compared that with what is seen in other approaches. 5.5.2 does this partially by comparing LLM-only and LLM in combination with model-based refactoring. In 4.3.2 there is discussion on making effective prompts based on quality attributes. The evaluation could consider the responses to the proposed prompts, and how effective the different attributes of the prompts are at producing useful outputs compared to other styles of prompts. It could assess the resulting suggestions, how often they are used or rejected, reasons why they were rejected, how easy it is for a human user to identify a suggestion from the agent as useful or not, and so on.

Reviewer: 2

Public Comments (these will be made available to the author)

The paper presents a method, called RapidOOD, which attempts to make use of LLM-based techniques combined with more traditional MDE techniques for generating design models from requirement models. The method makes use of a number of predefined prompts (on the LLM side), and OCL-based constraints (on the MDE side) to support design generation. A particularly interesting aspect of the paper is that it also explores the use of OO quality metrics to evaluate the results of generation.

There are a number of interesting concepts and techniques deployed in this paper, but I have to admit that I found the presentation confusing, possibly because there are too many ideas presented (and thus the story isn't clear), and it also wasn't entirely clear what is new in this paper, and what appears in the authors' previous work. I do note that the authors cite a number of their past papers, including work on tools, and I think they need to do a better job in distinguishing what is new here, and what is reused.

In my opinion, the two most interesting and novel parts of the paper are the "hybrid" approach to design generation, and the integration of quality metrics. In terms of the "hybrid" approach - it isn't entirely hybrid in one sense, because rule-based transformation is used for initial design generation, and LLMs are used for refactoring and improvement. I did not see an argument as to why this is the most suitable approach to use, beyond "degrees of freedom". Equally one could argue for LLMs as design space exploration tools - very much as search-based software engineering techniques can be used - with constraints thereafter used to narrow the explored design space down to something reasonable and practical. I think one way around this would be to actually compare a full MDE approach to RapidOOD as well as another variant that uses LLMs for design

space exploration, and then model transformation for refactoring.

I was intrigued by the integration and use of quality metrics, which is an interesting idea lending itself to automation. The metrics chosen are OK but they aren't the only metrics that could be considered - because quality is often externally defined by the stakeholder. Moreover, the hard-coded metrics (Section 5.4) could be considered arbitrary. Why this combination of attributes? Why not a different combination? There's no justification given. An alternative approach would be to define mechanisms for specifying and selecting quality metrics, and then providing an automated workflow for calculating them (or implement metrics selection and calculation as a service broker of some kind - i.e., metrics as a service!) This way, you aren't forcing yourselves to justify the choice of metrics and the way in which you combine them. This rather colours your experimental results because you are choosing the (a) initial models; (b) metrics that calculate success or failure; and (c) the training used to prepare the participants in your studies.

A few more detailed comments:

- I would remove all references to MDA. That is a specific OMG standard that involves a specific architecture and standards (MOF, QVT, etc). You are not using MDA, you are using MDE.
- The description of requirements model in Section 1 is very strange: (page 2, bullet point 1). Requirements do not always describe a system's external behaviour. They capture a stakeholders wants and needs. Possibly even their goals. A system model might capture external behaviour. This seems to indicate that you are actually prescribing constraints on your models in order for RapidOOD to work properly - and I think this needs to be made explicit early on.
- I would consider moving Section 2 to later, and get to the overall approach earlier. It's not clear to me that Section 2.1 is relevant to the paper: you assume that models are available to start with, so maybe consider removing this. Section 2.2 is more relevant and useful, but it's *very* focused on *generative* approaches - I cannot help but think a rather limited literature search was done here - and misses approaches that *transform* requirements models into design models. This has been the focus of, e.g., Kaindl's work amongst others. I also recommend looking at Tao Yue's survey of transformations from the Requirements Engineering journal in 2010.
- Section 3: this could perhaps be omitted, or shortened, so as to get to Section 4 earlier.
- Section 4: this really needs to bring the overall diagram earlier, and to make it very clear what parts are new, and what parts come from previous papers. It should also make it clear what the limitations are of having a prescribed architecture with a Service class and an EntityManager class. This is obviously a choice, but is it the only choice, and what are the downsides and benefits? Does it, for example, make refactoring easier later on?
- The use of OCL for specifying transformation rules is OK - but unorthodox. Why not use ATL, or Epsilon, or any other proper transformation language? As you note, you thus require end users to write OCL rules - is this easier than ATL or Epsilon rules (probably not).
- Section 5: the evaluation is interesting but the research questions are rather imprecisely formulated (I think this is a relatively easy fix, RQ1 for example might need to be several questions). However, the models used are small, self-selected, and there is no apparent external comparison with other approaches. As mentioned earlier, the use of metrics is interesting but if you are selecting the metrics used, and how they are used, and the examples, then it is difficult if not impossible to generalize from the results.
- It would be interesting to know more about the training that has been given to participants, as this will have some influence on the results of the experiments.
- It would also be useful to compare the use of the LLM prompts with refactoring tools (like Henshin or Epsilon) as all of this can quite easily be done directly with them (and arguably the Henshin rules are shorter to write than the prompts). This comes back to my earlier point that a comparison with a fully MDE solution would be worth considering.

-